



Memoria Cache

Scopo della cache: disaccoppiare le velocità di CPU e RAM, Località spaziale: alta probabilità di accedere in tempi successivi a indirizzi molto vicini, Località temporale: alta probabilità di accedere più volte agli stessi indirizzi in tempi molto vicini. Gerarchie di cache: a 2 o 3 livelli. Cache inclusive: ciascuna contiene quella del livello superiore

Lo spazio di memoria è organizzato in blocchi (da 4 a 64 byte), chiamati anche linee di cache. Le cache sono organizzate in righe (o slot): ciascuna contiene una linea di cache, cioè un blocco di memoria. Tutti i trasferimenti avvengono a livello di blocco. Quando una word (un dato) non viene trovata in cache, mi prendo dalla memoria centrale il dato e tutto il blocco. All'interno di un blocco ci possono essere più word (riga locazione cella della memoria principale) di memoria. Slot (blocco di memoria principale).

La cache per il programmatore è invisibile non vi si possono effettuare operazioni in programma macchina, Grazie alle località, l'accesso a locazioni contigue della memoria è più efficiente. Il numero di collisioni all'aumentare delle dimensioni del campo TAG aumenta. La politica di rimpiazzamento di linee LRU (Least Recently Used) ha senso solo nel caso di cache a più vie. In ogni tipo di cache, a ciascun blocco di memoria è associata una e una sola slot, quindi anche se la cache è a più vie il blocco di memoria può essere memorizzato in momenti diversi ma non in slot diversi.



- Indirizzi a 32 bit (spazio di indirizzamento di 2^{32} byte)
- Linee di cache (blocchi) di 32 byte
- Word di 4 byte
- Struttura dell'indirizzo:
 - I 27 bit più significativi rappresentano il numero di blocco
 - I successivi 3 bit il numero della word all'interno del blocco
 - Gli ultimi due bit il numero del byte all'interno della word

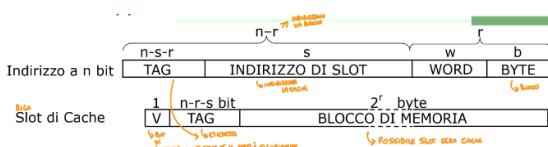
Ricerca di un blocco in cache

Una cache contiene un sottoinsieme di blocchi di memoria di indirizzo non contiguo. Quando la CPU cerca una word, non sa in quale posizione essa si possa trovare nella cache (se effettivamente c'è). Si utilizzano le Memorie Associative.

Memorie Associative

Ciascun elemento (riga) è costituito da due parti: la chiave e il dato. L'accesso ad un elemento viene effettuato non solo in base all'indirizzo ma anche in base a parte del suo contenuto (chiave). Nel caso di una cache: Un elemento viene chiamato slot di cache, La chiave viene chiamata tag (etichetta) L'informazione è una cache line (blocco).

Cache a Mappatura Diretta



Gestione della Cache

- La CPU deduce numero di slot e TAG del blocco a partire dall'indirizzo
- Se lo slot è valido, si confronta il TAG nella slot con quello del blocco
- Cache Hit in lettura: tutto ok
- Cache Hit in scrittura:
 - write through: scrive anche in memoria
 - write back: unica scrittura finale, quando il blocco è rimosso dalla cache
- Cache Miss in lettura: il blocco viene trasferito in cache e sovrascrive quello presente (questo va copiato in memoria se modificato)
- Cache Miss in scrittura:
 - write allocation: porta il blocco in cache (conviene per scritture ripetute)
 - write to memory: si effettua la scrittura in memoria

Spazio di memoria di 2^n byte, diviso in blocchi da 2^r byte. Gli $n-r$ bit più significativi dell'indirizzo specificano il blocco. In una cache con 2^s slot si associa ad ogni blocco la slot di cache indicata dagli s bit meno significativi del suo indirizzo. Se il blocco è in cache deve essere in quella slot, e lì bisogna cercarlo. Il TAG permette di distinguere tra tutti i blocchi che condividono la stessa slot (collidono)

Il meccanismo "write through" prevede che, ad ogni cache hit in scrittura, il dato scritto in cache venga scritto anche nel blocco di memoria corrispondente. Il meccanismo "write back", invece, prevede che ogni modifica a un blocco venga aggiornata in memoria solo nel momento in cui la slot associata al blocco modificato viene rimossa dalla cache. Una cache hit in scrittura non richiede necessariamente un accesso alla memoria.

Per gli esercizi sulle cache

Per trovare la dimensione minima della cache in grado di garantire che su ciascuna slot non collidano più di (TOT)K blocchi bisogna:

- Trovare la dimensione dell'etichetta (es. $32K = 2^5 \cdot 2^{10} = 2^{15}$ (dim. Etichetta))
- Trovare la dimensione di ciascun blocco (es. $64 \text{ byte} = 2^6$ (ciascun blocco 6 bit)), per capire quanti sono i bit per indirizzare gli slot, (es. 33 bit di indirizzi $33 - 15 - 6 = 12$ bit per indirizzare 2^{12} slot)
- Trovare poi la dimensione di ciascun slot (66 byte, 1 bit di validità, 15 bit di etichetta e 64 byte di blocco)
- Per cui la soluzione è $((64 + 2) \cdot 2^{12} = 264KB)$

Per massimizzare la dimensione della cache bisogna:

Tenere conto della grandezza dei blocchi (es. 64B), e capire quanto spazio abbiamo a disposizione sul microprocessore (es. 600KB). Ora voglio arrivare il più vicino possibile allo spazio a disposizione sul microprocessore infatti scompongo lo spazio a disposizione ($600KB = 600 \cdot 2^{10} = 75 \cdot 2^{13}$). Da questo vediamo i bit da dedicare agli slot, infatti vediamo che si può avere una cache avente 2^{13} slot di massimo 75 byte (avessimo scelto $150 \cdot 2^{12}$ non andava bene perché non massimizzava, avessimo tolto un 2 da 75 non andava bene perché i blocchi erano più piccoli di 64B). Ora che abbiamo i bit per indirizzare gli slot, non resta altro che trovare i bit di etichetta ($33 \text{ (bit totali)} - 13 \text{ (bit di slot)} - 6 \text{ (bit necessari per ciascun blocco)}$)

Per trovare il numero di collisioni nella stessa slot della cache bisogna:

Utilizzare l'etichetta infatti il numero di collisioni sarà dato da $2^{\text{etichetta}}$

Per trovare il numero massimo possibile di slot di una cache a due vie bisogna:

Trovare gli slot massimizzati della cache (vedi sopra), poi capire che quando aumentano le vie gli slot vengono diminuiti a seconda del numero di vie (es. cache a due vie la dimensione di ciascuna slot raddoppia, di conseguenza il numero di slot si deve dimezzare). (es. $2^{13} \cdot 2^{-1} = 2^{12} = 4096$)

Per calcolare il numero di slot e la posizione nella slot del byte con indirizzo esadecimale (xxxxx) bisogna:

Convertire l'indirizzo in binario, assegnare (trovando la dimensione di ciascun campo) a ogni bit il proprio campo. Per trovare le posizioni basta convertire in decimale i campi ricordandosi che gli indirizzi partono da tutti 0 (quindi solitamente bisogna aggiungere uno) (Es. (Tag: 12b Line: 14b Blocco: 6b 7B80034A Convertito 0111 1011 1000 0000 0000 0011 01 00 1010 14^0 posizione slot (line) 10^0 posizione blocco)

Per vedere se due byte di indirizzo esadecimale (XXXXX) e (XXXXX) collidono sulla stessa slot bisogna:

Convertire i due indirizzi in esadecimale e controllare che abbiano gli slot uguali ma etichetta diversa

Per avere la struttura della slot di cache bisogna:

Mettere 1b per la validità i bit per l'etichetta e i byte per il cache line, se la cache è a più vie va realizzato x (numero delle vie)

Per trovare la dimensione totale della cache bisogna:

Moltiplicare i byte degli slot per il numero di vie, sommargli i vari bit di validità e bit dell'etichetta (anche questi moltiplicati per il numero di vie se è a più vie) e successivamente moltiplicare per il numero di slot (es. 4 vie $16B \cdot 4 + 2^{12}$ (4K slot)).

I passi necessari alla ricerca nella cache del byte di indirizzo BXAXF2:

Scompongo l'indirizzo, accedo a un certo elemento della line, faccio una verifica dell'etichetta

Bus

In un'architettura a più bus vi sono diversi bus i quali soddisfano diverse esigenze come: Velocità di trasferimento, numero di linee, compatibilità all'indietro. La comunicazione sul bus è regolata da un protocollo di bus, in ciascun ciclo comunicano due soli dispositivi il master (periferica che collegata al bus fa iniziare un trasferimento) e lo slave (periferica che recepisce il trasferimento di dati). Quando parliamo di "larghezza" del bus intendiamo il numero di linee, solitamente vi sono le linee indirizzo e le linee per le operazioni, i bus hanno un problema ovvero al crescere della velocità del bus aumenta il bus skew (differenza nella velocità di propagazione dei segnali su linee diverse).

Segnali asseriti e negati

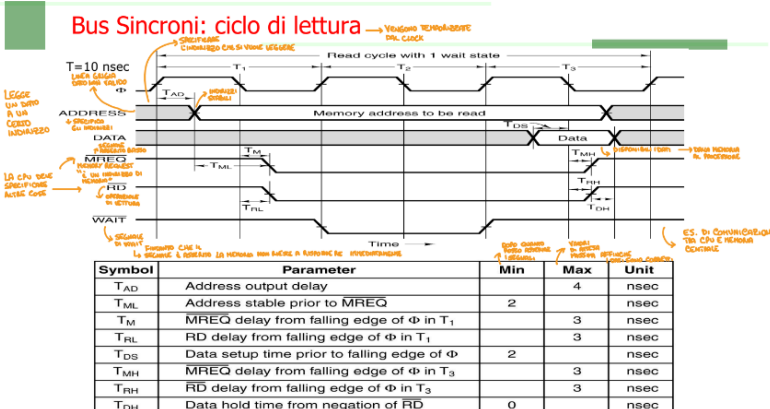
In alcuni casi un segnale provoca l'azione corrispondente quando la sua tensione è alta (1), in altri quando è bassa (0). Segnale asserito: quando assume il valore (alto o basso) che provoca l'azione, Segnale negato:

Si adotta la seguente notazione:

- **S**: segnale che è asserito alto
- **$\bar{S}$** : segnale che è asserito basso

altrimenti.

Bus sincroni: ciclo di lettura



In un bus sincrono una transazione richiede complessivamente sempre un tempo pari a un multiplo del periodo di clock.

I bus sincroni vengono temporizzati dal clock. Ci sono varie linee la prima è il periodo, la seconda serve a specificare l'indirizzo che si vuole leggere, quando è grigia significa che il dato non è valido, successivamente significa che sono stabili gli indirizzi. Poi c'è la linea dei dati, che è grigia fino a quando non sono disponibili i dati, successivamente MREQ (segnale asserito basso, memory request), RD (read per le operazioni di lettura), WAIT (Fintanto che il segnale è asserito la memoria non riesce a rispondere immediatamente)

Bus Sincrono: TempORIZZAZIONE

ES

Frequenza 100 MHz, periodo 10 nsec.

- primo vincolo: tempo a disposizione della memoria fra:

- la comparsa dell'indirizzo sul Bus
- la disponibilità dei dati sul Bus

$$\tau_1 = 2.5 \times T - T_{AD} - T_{DS} = 25 - 4 - 2 = 19 \text{ nsec}$$

(una memoria da 10 nsec ce la fa di sicuro)

- secondo vincolo: tempo a disposizione della memoria fra:

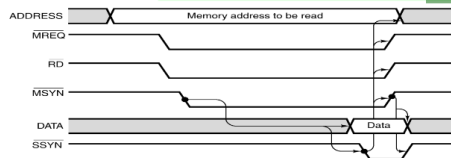
- l'asserzione di $\overline{\text{MREQ}}$ e $\overline{\text{RD}}$
- la disponibilità dei dati sul Bus

$$\tau_2 = 2 \times T - T_M - T_{DS} = 20 - 3 - 2 = 15 \text{ nsec}$$

Se il chip di memoria non soddisfa questi requisiti mantiene asserito il segnale di WAIT per introdurre stati di wait, cioè cicli di bus aggiuntivi.

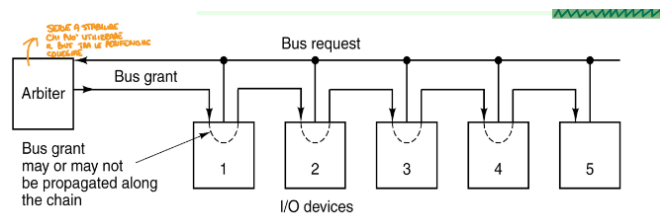
Bus asincrono: ciclo di lettura

Bus Asincrono: ciclo di lettura



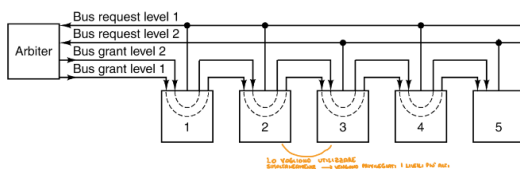
- Accoppiamento di dispositivi con velocità diverse
- Gli eventi avvengono in risposta ad altri eventi
- Full handshake:
 - MSYN asserted
 - SSYN asserted in response to MSYN when the data is ready
 - MSYN negated in response to SSYN
 - SSYN negated in response to the negation of MSYN

Arbitraggio del bus



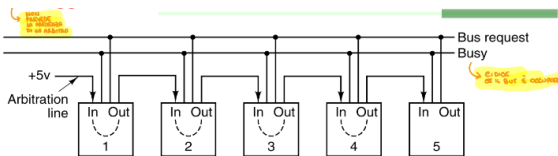
Permette di decidere quale dispositivo sarà il prossimo Bus Master risolvendo eventuali conflitti, spesso l'arbitro è nel chip del microprocessore, la linea di richiesta è condivisa, il bus grant è propagato dall'arbitro prima dell'inizio del ciclo. In questo tipo di arbitraggio non è vero che ogni dispositivo, in generale, ha un livello di priorità diverso, infatti, ogni dispositivo ha un livello di priorità uguale a tutti gli altri dispositivi collegati sulla stessa linea di richiesta

Livelli multipli di priorità



Diverse linee di richiesta associate a diversi livelli di priorità, in caso di conflitto favorite le catene a priorità più alta, all'interno di ciascuna catena vale la posizione, in genere se c'è un solo bus con anche la memoria, la CPU ha priorità più bassa dei dispositivi I/O (e.g. dischi)

Arbitraggio decentralizzato



Quando nessun dispositivo vuole il bus, la linea di arbitraggio è asserta con propagazione a tutti i dispositivi. Quando un dispositivo vuole il bus: invia una richiesta di bus, verifica se il bus è libero, se IN è asserto diventa master, nega Out e asserisce Busy, se In non è asserto non diventa master e nega Out.

Domande dell'homework

Sul bus PCI è possibile trasmettere fino a 64 bit in parallelo, il meccanismo di pipeline non ha nulla a che fare con il funzionamento di un bus. Sul bus PCI Express la velocità di comunicazione non è fissa ma dipende dal dispositivo che usa il bus. La comunicazione su bus USB è seriale. Non è possibile connettere più dispositivi sulla stessa linea USB

Per gli esercizi sui BUS sincroni

Come fare il diagramma di temporizzazione (bus sincroni):

Come prima cosa bisogna calcolare il periodo, che viene raffigurato come una linea che a metà periodo inizia a scendere per poi risalire, il tempo di risposta della memoria ci serve per capire quando è disponibile la linea dei dati. Ricorda: se i segnali sono asseriti bassi le linee vanno fatte al contrario. MREQ e RD partono a metà del primo periodo e si fermano nel momento in cui si inizia un'operazione di lettura (il fronte di lettura del clock si è la linea verticale a metà del periodo, infatti bisogna aspettare che i dati siano disponibili e successivamente iniziare la lettura al primo fronte disponibile). Il segnale di WAIT invece parte dopo il primo periodo e si ferma prima dell'ultimo periodo. La linea indirizzi invece parte disponibile dall'inizio e diventa grigia quando inizia la lettura.

Quanto deve essere asserito il segnale di WAIT per garantire una lettura in memoria?

Basta vedere sul grafico per quanto tempo viene asserito (alto o basso) il segnale di WAIT

In condizioni ideali, dopo quanto tempo i dati saranno disponibili presso il master?

Basta guardare il fronte di lettura del clock, guardo dopo quando sono disponibili i dati e cerco il primo fronte di lettura disponibile

In condizioni ideali, quanto dura, complessivamente, una transazione di lettura?

Basta guardare tutti i periodi che vengono utilizzati ricordandosi che i segnali di controllo non possono essere asseriti prima di un nuovo fronte del segnale di clock, corrispondente all'inizio del periodo successivo.

In condizioni ideali, quanto dura, complessivamente, una transazione di lettura se la frequenza si dimezza?

Basta rifare il calcolo di prima ragionando con la durata del clock raddoppiata.

In condizioni ideali, quanto dura, complessivamente, una transazione di lettura se la frequenza raddoppia?

Stesso calcolo di prima con durata del clock dimezzata.

Dopo quanto tempo i dati saranno disponibili presso il master se si lavora a 50 MHz e la memoria ha un tempo di risposta di 60 ns, tenendo conto del fatto che il master inserisce gli indirizzi con un ritardo di 5 ns al massimo?

Questa domanda vuole sapere quando inizia la lettura, il calcolo da fare è sempre lo stesso bisogna solo fare attenzione al ritardo inserito, che ritarda anche la disponibilità dei dati di 5ns.

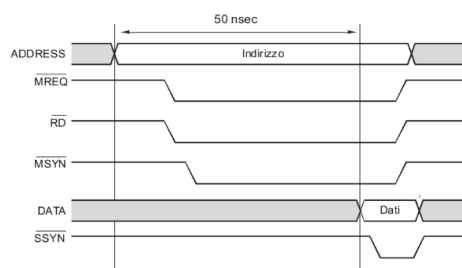
Per gli esercizi sui BUS asincroni

Generalmente non vi sono domande sui BUS asincroni, comunque:

In un bus asincrono, i segnali MSYN (Master Synchronize) e SSYN (Slave Synchronize) sono utilizzati per il controllo del trasferimento dei dati tra dispositivi, in particolare per coordinare la comunicazione tra un dispositivo master (controllore) e un dispositivo slave (periferica). MSYN (Master Synchronize): Questo segnale viene inviato dal dispositivo master per indicare l'inizio di una transazione. Quando il master vuole iniziare un trasferimento di dati, attiva il segnale MSYN. Questo segnala allo slave che il master è pronto a iniziare il trasferimento dei dati. SSYN (Slave Synchronize): Questo segnale viene inviato dal dispositivo slave in risposta al segnale MSYN del master. Una volta che lo slave è pronto per ricevere o trasmettere i dati, attiva il segnale SSYN per indicare al master che può procedere con il trasferimento dei dati.

Per tracciare il diagramma di temporizzazione:

Solitamente sono tutti uguali, quello che può cambiare è il tempo di risposta della memoria che ci dice quando sono disponibili i dati. Solitamente gli indirizzi sono disponibili dopo poco e si fermano poco prima della fine dell'ultimo periodo, successivamente vi sono MREQ e RD che sono esattamente uguali, poi vi è MSYN che parte poco dopo i precedenti due e finisce insieme ai precedenti due (finiscono più o meno quando finisce la linea indirizzi), poi vi sono i dati che sono disponibili a seconda del tempo di risposta della memoria e poi vi è il segnale SSYN che inizia quando inizia la lettura dei dati.



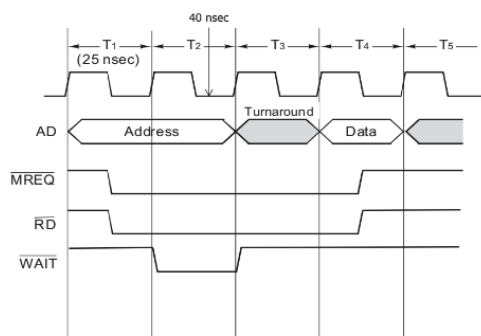
Per gli esercizi su BUS PCI

Anche per questi generalmente non vi sono domande

Per tracciare il diagramma di temporizzazione (bus sincroni):

Il periodo funziona nella stessa identica maniera di quando non vi sono linee condivise, anche MREQ e RD funzionano ugualmente (partono a metà del primo periodo, e finiscono quando inizia l'operazione di lettura). Ciò che cambia è la linea indirizzi che viene indicata con AD infatti ora è condivisa sia per gli indirizzi che per i dati, lo spazio per gli indirizzi termina alla fine del periodo in cui iniziano ad essere disponibili i dati, infatti successivamente vi sarà un periodo di turnaround (Infatti essendo la linea condivisa il master avvia la comunicazione e guida le linee di indirizzo, dopo segue un cambio di controllo (turnaround), il quale impiega un ciclo e dopodichè passa il comando allo slave per una lettura dei dati), Successivamente quindi la linea diventa per i dati e la lettura avverrà al primo fronte di lettura disponibile.

Es.



Ci sono due tipi di modellazione: La parte del data path è pressochè identica, ciò che cambia sono le richieste dell'esercizio.

Per le architetture RISC:

Si parte dal data path, a sinistra del data path si mette la RAM, si mettono a partire dal basso (ovviamente a seconda delle specifiche del problema): un registro accumulatore (è comodo e rende l'esercizio più semplice), i registri di uso generale, i registri richiesti come ad esempio PC (program counter) e IR (instruction register), se vi è richiesta di registri per il trasferimento dei dati da/per la memoria vengono messi (a partire dal basso MDR e MAR). Dopo aver terminato il data path faccio dei collegamenti con la ram, MAR ha una freccia uscente, MDR ha una freccia doppia, PC ha una freccia uscente e IR ha una freccia entrante. Successivamente metto la ALU e la collego al registro accumulatore (BUS A) poi creo un bus che collega tutti i registri e finisce sotto la ALU che è il bus c successivamente ne creo un altro che è la seconda entrata della ALU (BUS B) a cui collego tutti i registri a eccezione di quello accumulatore e il MAR. Ora viene fatta la parte di controllo che prevede un registro dalla quale partono tutti i segnali di controllo, una parte gestisce la ALU la quale avrà una specifica (ad es. se deve gestire 16 operazioni saranno necessari 4 bit $2^4=16$), da questa zona partirà una freccia che collegherà la ALU, un'altra parte è per il controllo dei registri bus C, le frecce uscenti saranno tante quante sono i collegamenti con il BUS, poi serve una parte per il controllo della memoria (se la traccia richiede fetch delle istruzioni, lettura e scrittura saranno necessari 3b) da qua partirà una freccia che collegherà MAR. Successivamente è necessaria un'ultima parte per il controllo del bus B, per farlo è necessario un decodificatore il quale deve avere un numero di ingressi che contenga tutti i registri collegati al BUS B (seguendo questo esempio abbiamo fatto 5 collegamenti quindi $2^2=4 < 5 < 2^3$ sarà un decodificatore a 8 ingressi), collego il decodificatore a tutti i registri che erano collegati al bus B (in entrata invece faccio uscire il numero di bit utilizzati (3)). Infine dispongo un instruction decoder sopra i segnali di controllo faccio tutti i collegamenti e mando una freccia da IR.

Per le architetture CISC:

La parte iniziale dove metto la RAM, il data path, la ALU, il BUS B e il BUS C è uguale, anche la parte dei segnali di controllo è simile, infatti per la ALU si ragiona alla stessa maniera, per il bus c uguale solo che si mette una sola freccia, per la parte della memoria il discorso è uguale, stessa cosa per la gestione del controllo del BUS B aggiungo un decoder e faccio i collegamenti. La parte nuova consiste nell'aggiungere un dispositivo detto Control store che si occupa di immagazzinare le microistruzioni, per farlo è necessario aggiungere un altro registro rinominato (MPC MicroProgramCounter) che indirizza una certa microistruzione, per farlo collego l'IR al MPC che collega poi il control store che a sua volta manda una freccia ai segnali di controllo, aggiungo un campo tra i segnali di controllo che rinomineremo (ADD) di address (serve a capire la prossima microistruzione), che collego a MPC. Attenzione: i bit uscenti da questo campo cambiano a seconda delle microistruzioni del CS (es. 512 microistruzioni 9 bit $2^9=512$)

Indicare possibili modifiche dell'architettura proposta in grado di migliorare le prestazioni

Si potrebbe introdurre un nuovo bus, una pipeline per un parallelismo a livello di istruzioni

Architettura che non possiede RAM ma solo ROM:

Si può raffigurare come un'architettura RISC o CISC a seconda della specifica, quello che cambia è che al posto della RAM ci sta la ROM che ha gli stessi collegamenti, poi cambia la gestione della memoria, infatti ci sarà un campo aggiuntivo nei segnali di controllo (I/O) per la gestione di input e output dal quale uscirà un collegamento verso un nuovo dispositivo (I/O) il quale si occuperà di lettura e scrittura (essendo due operazioni + 1 (quella in cui non fa nulla) avrà bisogno di almeno 2 bit $2^2=4 > 3$) quest'ultimo avrà una doppia freccia con il bus B, vi sarà poi un campo aggiuntivo nei segnali di controllo per il fetch che effettuerà un collegamento con IR.

Architettura con IFU e Pipeline a più stadi:

Si raffigura l'architettura CISC o RISC a seconda della specifica, successivamente si mette IFU tra registri (solo tra PC e IR) e memoria, la indico come primo stadio, poi aggiungo delle "frontiere" per individuare gli altri stadi, (es. pipeline a 4 stadi: parte dell'IFU primo stadio, aggiungo una frontiera sul bus B per individuare il secondo stadio, ne metto una sul bus C per il 3 stadio e un'altra sempre su C per il 4).

Assembly

Per interpretare il programma utilizziamo un tracer, è un interprete che procede passo-passo. I registri che vengono utilizzati sono 4 (AX, BX, CX, DX) che sono a loro volta divisi in due parti (es. AH AL).

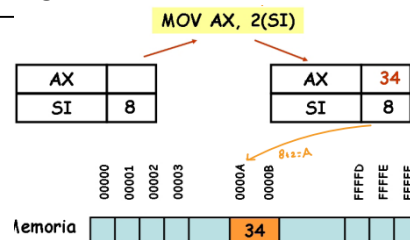
AX: registro accumulatore, usato per memorizzare il risultato dell'elaborazione e come destinazione di molte istruzioni (Es. ADD AX, 20) (Prendo AX ci sommo 20 e rimetto tutto dentro quello che c'è a sinistra della virgola (AX))

BX: registro base, usato (indirizzamento alla memoria principale) come accumulatore o come puntatore alla memoria (Es. MOV AX, (BX) se è tra parentesi tonde è una locazione di memoria principale (viene detto indirizzo effettivo). Prendi il contenuto di BX sommalo a AX e metti il dato in AX)

CX: registro contatore, usato come contatore dei cicli (operazioni in generale)

DX: Registro dati, usato (stringhe) insieme ad AX per contenere le istruzioni lunghe due parole (32 bit)

Quando c'è un numero davanti al registro, l'indirizzo si ottiene come somma del contenuto più una costante.



Ogni programma assembly è strutturato in 3 sezioni:

- Sezione di TESTO: (direttiva .SECT .TEXT): contiene le istruzioni del programma
- Sezione DATI (direttiva .SECT .DATA): alloca spazio nel segmento DATI per i dati (inizializzati)
- Sezione BSS (direttiva .SECT .BSS): la lasceremo sempre vuota

Si possono definire etichette di due tipi:

Globali: identificatori alfanumerici seguiti dal simbolo ":" (possono occupare una intera riga)

Locali: utilizzabili solo nel segmento TESTO, costituite da una sola cifra seguita dal simbolo ":"

```
!calcolo di (a+3)*b
_EXIT = 1
.SECT .TEXT
start: Punto di inizio del programma
    MOV AX,(a)
    ADD AX,3
    MUL (b)
    PUSH 0
    PUSH _EXIT
    SYS
.SECT .DATA Mappa della memoria
a: .WORD 5
b: .WORD 3
.SECT .BSS Sezione che lasceremo sempre vuota
```

Se inizia col punto esclamativo è un commento

Prende la locazione di memoria etichettata con a. Metti il contenuto di a in AX. Prendi AX e sommi 3, metti tutto in AX. la mul moltiplica sempre per AX, per terminare il programma bisogna invocare delle routine di sistema operativo le quali hanno un identificatore. EXIT rilascia tutta la memoria utilizzata. Per invocare una funzione di sistema operativo bisogna utilizzare la pila, metti prima 0 poi la funz di routine 1 (exit) gli avevo precedentemente assegnato un'etichetta. A questo punto devo invocare la funzione di sistema senza argomenti, infatti il sistema andrà a cercare gli argomenti sulla pila

Metti la costante 5 in a, Metti la costante 3 in b

Per terminare un programma bisogna utilizzare la pila che serve a invocare una funzione di sistema operativo, prima metto 0 e poi 1 (dove _EXIT sarebbe l'etichetta data a 1)

Operazioni

ADD(B): somma all'operando destinazione e memorizza il risultato nell'operando destinazione.

SUB(B): sottrae l'operando sorgente all'operando destinazione e memorizza il risultato nell'operando destinazione.

(I)MUL(B): moltiplica due operandi (Il primo operando è sempre il registro accumulatore), il secondo va specificato.

(I)DIV(B): come MUL per la divisione. Se source (il registro che inseriamo noi) ha dimensioni 1 byte, il risultato va in AL, il resto in AH. Se source ha dimensioni 1 word, il risultato va in AX e il resto in DX

AND(B), OR(B), XOR(B): si fanno con due termini

SHR(B), SHL(B), ROL(B), ROR(B): principali operazioni di scorrimento

Salti incondizionati (specifichiamo noi)

JMP: trasferisce il controllo all'istruzione specificata dall'operando in maniera incondizionata

Salto Corto: la destinazione si trova nel segmento di codice corrente (cui fa riferimento il registro CS)

Salto Lungo: l'istruzione modifica il contenuto del registro CS

CMP(B): Effettua una sottrazione fra due operandi senza modificare

Salti condizionati

Jxx: istruzioni di salto in base ai valori del registro di flag

Le istruzioni di salto condizionato controllano se una certa condizione è verificata.

Se la condizione è verificata, il controllo passa all'istruzione il cui indirizzo è specificato come operando.

Se la condizione non è verificata, l'esecuzione prosegue con l'istruzione successiva.

```

If (a > b) then
    a=b;
else
    b=a;

while (a<1000)
    ...
end while;

CMP AX,BX
JLE else_label
MOV AX,BX
JMP end_label

else_label:
    MOV BX,AX
end_label:
    ....

whileSum:
    CMP AX,1000
    JGE end_while
    ...
    JMP whileSum
end_while:
    
```

Faccio compare AX e BX ho un ramo vero e uno falso, implemento prima il ramo else (dove non è verificata, quando a <= a b),

Ciclo che va avanti per mille volte: scrivo un'etichetta, scrivo tutte le istruzioni del contenuto del ciclo, se A è più grande di 1000 faccio il salto (JGE) (salto a una istruzione più avanti), senno alla fine ho un salto all'indietro che va a rifare questo confronto

Instruction	Description	When to jump
JNA, JBE	Below or equal	CF=1 or ZF=1
JNB, JAE, JNC	Not below	CF=0
JE, JZ	Zero, equal	ZF=1
JNLE, JG	Greater than	SF=OF and ZF=0
JGE, JNL	Greater equal	SF=OF
JO	Overflow	OF=1
JS	Sign negative	SF=1
JCXZ	CX is zero	CX=0
JB, JNAE, JC	Below	CF=1
JNBE, JA	Above	CF=0&ZF=0
JNE, JNZ	Nonzero, nonequal	ZF=0
JL, JNGE	Less than	SF=OF
JLE, JNG	Less or equal	SF=OF or ZF=1
JNO	Nonoverflow	OF=0
JNS	Nonnegative	SF=0

Ciclo che va avanti per mille volte: scrivo un'etichetta, scrivo tutte le istruzioni del contenuto del ciclo, se A è più grande di 1000 faccio il salto (JGE) (salto a una istruzione più avanti), senno alla fine ho un salto all'indietro che va a rifare questo confronto

L'istruzione LOOP consente di implementare esplicitamente cicli, Il registro CX deve essere inizializzato con il numero di cicli dell'istruzione LOOP, quando incontri quest'istruzione, prende il contenuto di CX ci toglie 1 se il risultato ottenuto è 0 fa l'istruzione successiva, senno salto all'etichetta.

```

for (i=0; i<5; i++) {
    MOV AX,(a)
    MOV CX,5
    repeat:
        ADD AX,3
        LOOP repeat
}
    
```

Domande che sono capitate:

@SI L'i7 ha un'architettura superscalare

@NO Un microprocessore CISC non può avere una architettura superscalare

@SI L'i7 ha una architettura ibrida in parte CISC e in parte RISC

@SI ARM \ ' è una architettura implementata su diversi microprocessori

@SI L'ATmega168 lavora a frequenze dell'ordine dei Mhz

@SI L'ATmega168 \ ' è un microprocessore RISC

@SI L'ATmega168 ha una pipeline a due stadi

QUARTO HOMEWORK

INFO.

BUS SINCRONO 100 MHz

LINEA DATI

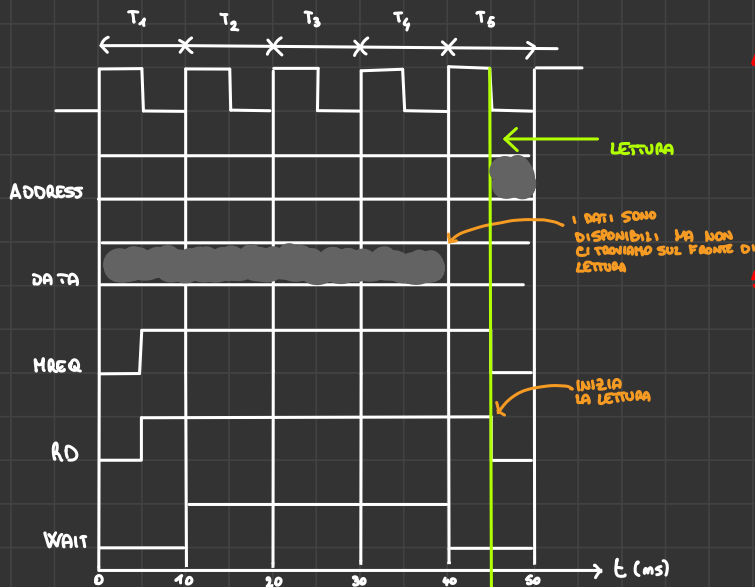
LINEA INDIRIZZO

MREQ, RD E WAIT

MEMORIA CON UN TEMPO DI RISPOSTA DI 40 ns

DIAGRAMMA DI TEMPORIZZAZIONE

$$\frac{1}{100 \text{ MHz}} = \frac{1}{100 \cdot 10^6} = \frac{1}{100} \cdot 10^{-6} = \frac{1000}{100} \cdot 10^{-9} = 10 \text{ ns}$$



1.

Il segnale di WAIT è asserito da $t = 10 \text{ ns}$ a $t = 40 \text{ ns}$ quindi $40 - 10 = 30 \text{ ns}$

2.

BASTA GUARDARE IL FRONTE DI LETTURA 45 ns

3.

$5 \cdot 10 = 50 \text{ ns}$

Infatti, dopo che i dati vengono letti dal master a metà del periodo T5, i segnali di controllo non possono essere asseriti prima di un nuovo fronte del segnale di clock, corrispondente all'inizio del periodo successivo. Pertanto una transazione di lettura dura 5 periodi.

4.

Ogni periodo dura 20 ns quindi i dati sono disponibili all'inizio del terzo periodo, la lettura inizia a 50 ns, si aspetta completamente la fine dell'ultimo periodo del clock e di conseguenza ci si impiega 60 ns

5.

Ogni periodo dura complessivamente 5 ns di conseguenza, bisogna aspettare l'inizio del 9 periodo per far sì che i dati siano disponibili la lettura inizierà a 42,5 ns e si aspetterà la fine dell'ultimo periodo di clock, ci si impiegherà complessivamente 45 ns

6.

$$T = \frac{1}{50 \cdot 10^6} = \frac{1000}{50} \cdot 10^{-9} = 20 \text{ ns}$$

La lettura del dato inizia a 50ns, il segnale di wait rimane asserted da 20ns a 40ns.

$$40 - 20 = 20 \text{ ns}$$

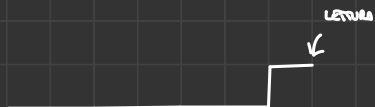
7.

$$T = 20 \text{ nSEC}$$

$$\text{DATI} = 60 \text{ nSEC}$$

$$\text{INDIRIZZ1} = 5 \text{ nSEC R.}$$

$$70 \text{ ns}$$



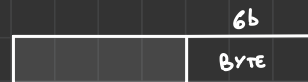
QUINTO HOMEWORK

INTRO.

Si vuole progettare una cache a mappatura diretta per un sistema a 64 bit per una memoria da 8GB (quindi sono solo 33 i bit utili per gli indirizzi) e blocchi di 64 byte. Rispondere alle domande che seguono supponendo di avere a disposizione uno spazio di 600KB sul microprocessore.

QUESTO PERCHÉ
 $8GB = 2^{33}$ QUINDI
 SERBONO 33 BIT
 PER INDIRIZZARE LA
 MEMORIA

STRUTTURA INDIRIZZO



STRUTTURA CACHE



1.

Indicare la dimensione minima della cache in grado di garantire che su ciascuna slot non collidano più di 32K blocchi

DA QUA PRENDO L'ETICHETTA

$32K = 2^5 \cdot 2^{10} = 2^{15} \rightarrow$ È POSSIBILE GARANTIRE UN MASSIMO DI 32K COLLISIONI SU CIASCUNA SLOT CON UN ETICHETTA DI 15 BIT

DA QUA PRENDO 1 BIT DEL BLOCCO

$64 = 2^6 \rightarrow$ È POSSIBILE SPECIFICARE UN BYTE DI CIASCUN BLOCCO CON 6 BIT

CON INDIRIZZI DI 33 BIT RIMANGONO $33 - 15 - 6 = 12$ BIT PER INDIRIZZARE 2^{12} SLOT

CIASCUNA SLOT È DUNQUE FORMATA COMPLESSIVAMENTE DA 66 BYTE (1 BIT DI VALIDITÀ, 15 BIT DI ETICHETTA E 64 BYTE DI BLOCCO), PER CUI LA DIMENSIONE MINIMA DELLA CACHE IN GRADO DI GARANTIRE CHE SU CIASCUNA SLOT NON COLLIDANO PIÙ DI 32K BLOCCHI È

$(64 + 2) \cdot 2^{12} = 264 \text{ KB}$

$m = 33 \quad s = 12 \quad b = 6$



2.

Indicare, indipendentemente dal numero di collisioni, la struttura dell'indirizzo che consente di massimizzare la dimensione della cache specificando la dimensione dei vari campi (tag, line, byte)

VOGLIO ARRIVARE IL PIÙ VICINO POSSIBILE A 600KB, AUMENTO LE SLOT AUMENTANDO COSÌ IL NUMERO DI BLOCCHI CONTENIBILI IN TOTALE AGGIUNGO 1 BIT ALE SLOT E TOLGO 1A TAG (SOMMA TOTALE 33)



MASSIMIZZO PERCHÉ SE POSSO $2^{12} \cdot 150$ NON MASSIMIZZEREBBE E SE PROCOGLIERO UN ALTRO 2 SCENDEREI SOTTO 64

$600 \text{ KB} = 600 \cdot 2^{10} = 75 \cdot 2^{13}$ FACENDO USO DI TUTTO LO SPAZIO DISPONIBILE SUL MICROPROCESSORE, È POSSIBILE AVERE UNA CACHE CON 2^{13} SLOT DI MASSIMO 75 BYTE

$64 = 2^6 \rightarrow$ È POSSIBILE SPECIFICARE UN BYTE DI CIASCUN BLOCCO 6 BIT

CON INDIRIZZI DI 33 BIT RIMANGONO $33 - 13 - 6 = 14$ BIT DI ETICHETTA

3.

Indicare la struttura di una slot della cache individuata nella domanda precedente, specificando la dimensione dei vari campi (validità, tag, dati)

1b: 16b: 64b

4.

Indicare il numero di collisioni nella stessa slot della cache avente la struttura individuata nella domanda precedente

$$2^{14} = 16384$$

5.

Indicare, indipendentemente dal numero di collisioni, il numero massimo possibile di slot di una cache a due vie.

$$2^{13} \cdot 2^{-1} = 2^{12} = 4096$$

Esercizio su Memoria Cache I

CACHE A MAPPA DIRETTA CON 16K SLOT

CACHE LINE DI 64 BYTE

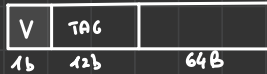
INDIRIZZI A 32 BIT

1.

$$16K = 2^4 \cdot 2^{10} = 2^{14} \text{ SLOT}$$

$$64B = 2^6 \text{ 6 BIT PER BLOCCO}$$

$$TAG = 32 - 14 - 6 = 12$$



2.



SLOT = $8 + 4 + 1 = 13$ DATO CHE IL PRIMO INDIRIZZO È TUTTI 0 LA POSIZIONE È LA 14°

$$\text{Blocco} = 8 + 2 = 10 + 1 = 11^{\circ}$$

ESERCIZIO SU MEMORIA CACHE II

$$4K \text{ SLOT} = 2^2 \cdot 2^{10} = 2^{12}$$

24 BIT INDIRIZZI

$$\text{CACHE LINE 16 BYTE} = 2^4$$

$$\text{TAG} = 24 - 4 - 12 = 8$$

1.

TAG	SLOT	LINE
8b	12b	4b

2.

V	TAG	
1b	8bit	16b

 TUTTO x 4

3.

$$\text{DIM SLOT} = 16 \cdot 4 = 64\text{B} + 4\text{B} \xrightarrow{\text{L'ETICHETTA VALIDITA'}} + 0,5\text{B} \xrightarrow{\text{L'ETICHETTA VALIDITA'}} = 68,5\text{B}$$

$$\text{CACHE} = 68,5 \cdot 2^{12}$$

ESERCIZIO SU MEMORIE CACHE IV

INDIRIZZI 32 BIT

LINEE DI CACHE 32 BYTE

MEMORIA 4 MB SPAZIO DISPONIBILE 40 KB

E SERCIZIO SU BUS

BUS SINCRONO $100 \text{ MHz} = 0,1 \text{ GHz} = \frac{1}{0,1} = 10 \text{ nsec}$

MEMORIA 15 nsec (DEVI ASPETTARE LA FINE DEL PERIODO) $\rightarrow$ IL MASTER È PRONTO A PRENDERE I DATI A METÀ DEL PERIODO

QUINDI SI ASPETTA IL SECONDO PERIODO DATO CHE $(10 \angle 15 \angle 20)$

IL FRONTE DI DISCESA VALE LA METÀ DI UN PERIODO

E SI PRELEVA A METÀ DEL SECONDO PERIODO $(20 \angle 25 \angle 30)$

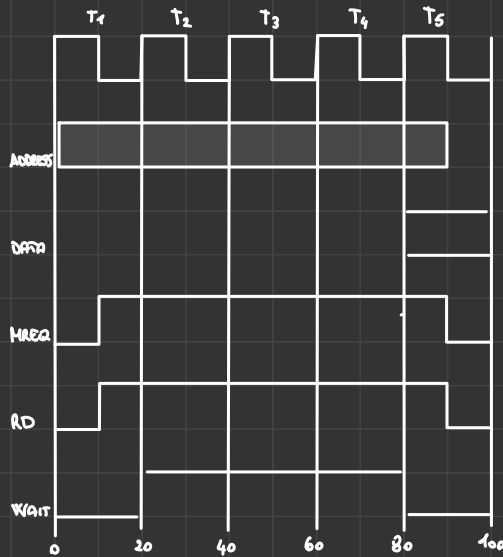


Seconda prova parziale 2019

1. Esercizio su bus. Si consideri un bus sincrono che lavora a 50 Mhz con linee separate per dati e indirizzi dotato dei segnali di controllo MREQ, RD e WAIT e una memoria con un tempo di risposta di 80 nsec dal momento in cui gli indirizzi sono stabili.
 - a) In condizioni ideali, per quanto tempo deve essere asserito il segnale di WAIT per garantire una lettura in memoria? Esprimere il risultato con un numero che indica il tempo in nsec.
 - b) In condizioni ideali, dopo quanto tempo i dati saranno disponibili presso il master? Esprimere il risultato con un numero che indica il tempo in nsec.
 - c) In condizioni ideali, quanto dura, complessivamente, una transazione di lettura? Esprimere il risultato con un numero che indica il tempo in nsec.
 - d) In condizioni ideali, quanto dura, complessivamente, una transazione di lettura se la frequenza si dimezza? Esprimere il risultato con un numero che indica il tempo in nsec.
2. Esercizio su cache. Si vuole progettare una cache a mappatura diretta per un sistema a 22 bit e blocchi di 32 byte. Rispondere alle domande che seguono supponendo di avere a disposizione uno spazio di 100KB sul microprocessore.
 - a) Indicare la dimensione minima della cache in grado di garantire che su ciascuna slot non collidano più di 128 blocchi. Esprimere il risultato nel formato XM (dove M=B,KB,MB,GB o TB) arrotondando all'intero superiore (per esempio: 125KB).
 - b) Indicare la struttura dell'indirizzo per la cache individuata nella domanda precedente, specificando la dimensione dei vari campi (tag,line,byte) nel formato X:Y:Z (per esempio: 5:12:8).
 - c) Indicare la struttura di una slot della cache individuata nella domanda precedente, specificando la dimensione dei vari campi (validità,tag,dati) nel formato XM:YM:ZM, dove M = b (bit) o B (Byte) (per esempio: 3b:15b:55B).
 - d) Indicare, indipendentemente dal numero di collisioni, la struttura dell'indirizzo che consente di massimizzare la dimensione della cache specificando la dimensione dei vari campi (tag,line,byte) nel formato X:Y:Z (per esempio: 5:12:8).
 - e) Indicare il numero di collisioni nel caso in cui, mantenendo lo stesso numero di slot della cache individuata nelle domande precedenti, la cache viene realizzata a due vie.
3. Scrivere un programma in linguaggio assembler 8088 che, dato un vettore v di 5 elementi in memoria principale, stampa il suo contenuto nell'ordine inverso (dall'ultimo al primo elemento).

1. Esercizio su bus. Si consideri un bus sincrono che lavora a 50 Mhz con linee separate per dati e indirizzi dotato dei segnali di controllo MREQ, RD e WAIT e una memoria con un tempo di risposta di 80 nsec dal momento in cui gli indirizzi sono stabili.
- a) In condizioni ideali, per quanto tempo deve essere assertito il segnale di WAIT per garantire una lettura in memoria? Esprimere il risultato con un numero che indica il tempo in nsec.
 - b) In condizioni ideali, dopo quanto tempo i dati saranno disponibili presso il master? Esprimere il risultato con un numero che indica il tempo in nsec.
 - c) In condizioni ideali, quanto dura, complessivamente, una transazione di lettura? Esprimere il risultato con un numero che indica il tempo in nsec.
 - d) In condizioni ideali, quanto dura, complessivamente, una transazione di lettura se la frequenza si dimezza? Esprimere il resultato con un numero che indica il tempo in nsec.

$$T = \frac{1}{50 \cdot 10^6} = \frac{1000}{50} \cdot 10^{-9} = 20 \text{ ns}$$



③ 60 ns

③ 80 ns

③ 100 ns

④ 120 ns

2. Esercizio su cache. Si vuole progettare una cache a mappatura diretta per un sistema a 22 bit e blocchi di 32 byte. Rispondere alle domande che seguono supponendo di avere a disposizione uno spazio di 100KB sul microprocessore.
- a) Indicare la dimensione minima della cache in grado di garantire che su ciascuna slot non collidano più di 128 blocchi. Esprimere il risultato nel formato XM (dove M=B,KB,MB,GB o TB) arrotondando all'intero superiore (per esempio: 125KB).
 - b) Indicare la struttura dell'indirizzo per la cache individuata nella domanda precedente, specificando la dimensione dei vari campi (tag,line,byte) nel formato X:Y:Z (per esempio: 5:12:8).
 - c) Indicare la struttura di una slot della cache individuata nella domanda precedente, specificando la dimensione dei vari campi (validità,tag,dati) nel formato XM:YM:ZM, dove M = b (bit) o B (Byte) (per esempio: 3b:15b:55B).
 - d) Indicare, indipendentemente dal numero di collisioni, la struttura dell'indirizzo che consente di massimizzare la dimensione della cache specificando la dimensione dei vari campi (tag,line,byte) nel formato X:Y:Z (per esempio: 5:12:8).
 - e) Indicare il numero di collisioni nel caso in cui, mantenendo lo stesso numero di slot della cache individuata nelle domande precedenti, la cache viene realizzata a due vie.

22 BIT

$$32 \text{ BYTE BLOCCO} = 2^5$$

$$100 \text{ KB A DISPOSIZIONE} = 100 \cdot 2^{10} = 50 \cdot 2^{11}$$

$$\textcircled{a} 128 = 2^7$$

$$22 - 7 - 5 = 10$$

$$7: 10: 5$$

$$1b: 4b: 32B = 33B \cdot 2^{10} = 33KB$$

$$\textcircled{c} 4: 10: 5$$

$$\textcircled{d} 1b: 4b: 32B$$

$\textcircled{e}$ DISTRIBUIAMO 116 PER GLI SLOT

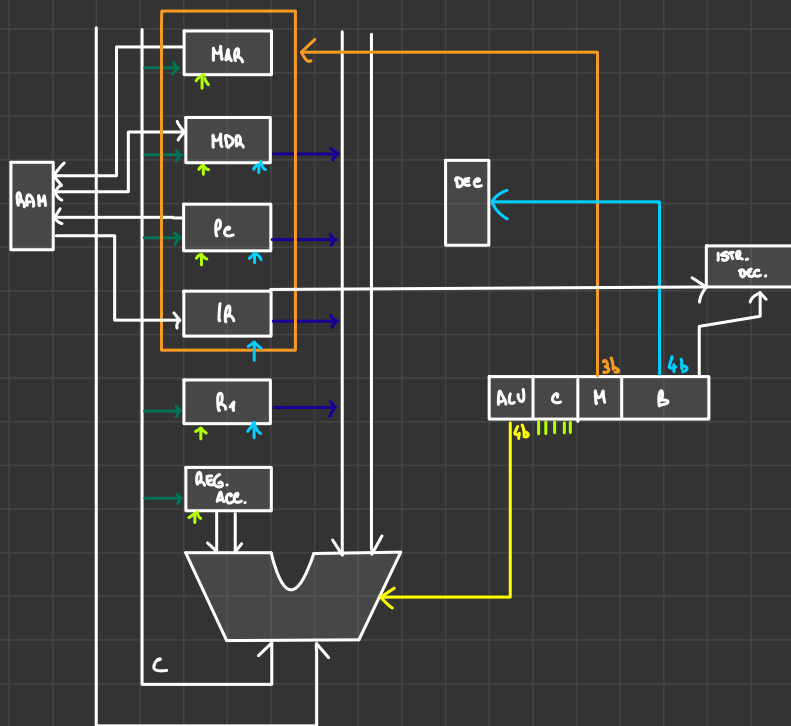
$$22 - 11 - 5 = 6$$

$$6: 11: 5$$

$$33B \cdot 2^{11} = 66KB$$

$$\textcircled{a} 2^6 = 64$$

ARCHITETTURA RISC



2021

$$T = \frac{1}{20 \cdot 10^6} = \frac{1000}{20} \cdot 10^{-9} = 50 \text{ ns}$$



ESEACIZIO SU CACHE

64 BIT

$$\text{RAM } 268 = 2 \cdot 2^{30} = 2^{31} \quad 2 \cdot 2^{30} = 2^{31}$$

CACHE DATI 4 VIE 2k SLOT

$$\text{CACHE } 328 = 2^5$$

CACHE ISTRUZIONI 8 VIE

LINE

$$\textcircled{3} 31 - 11 - 5 = 15$$

$$15:11:5$$

$$\textcircled{4} 16:16:328 = 328 \cdot 4 = 1288$$

$$\textcircled{4} 16:16:328, 16:16:328, 16:16:328, 16:16:328$$

$$\textcircled{4} 2^{15} \cdot 2^2 = 2^{17}$$

$$\textcircled{4} 2^{13} \cdot 2^3 = 2^{16} = 1k$$

$$\textcircled{4} 200 \text{ HB} = 200 \cdot 2^{20} = 100 \cdot 2^{21}$$

$$31 - 21 - 5 = 5$$

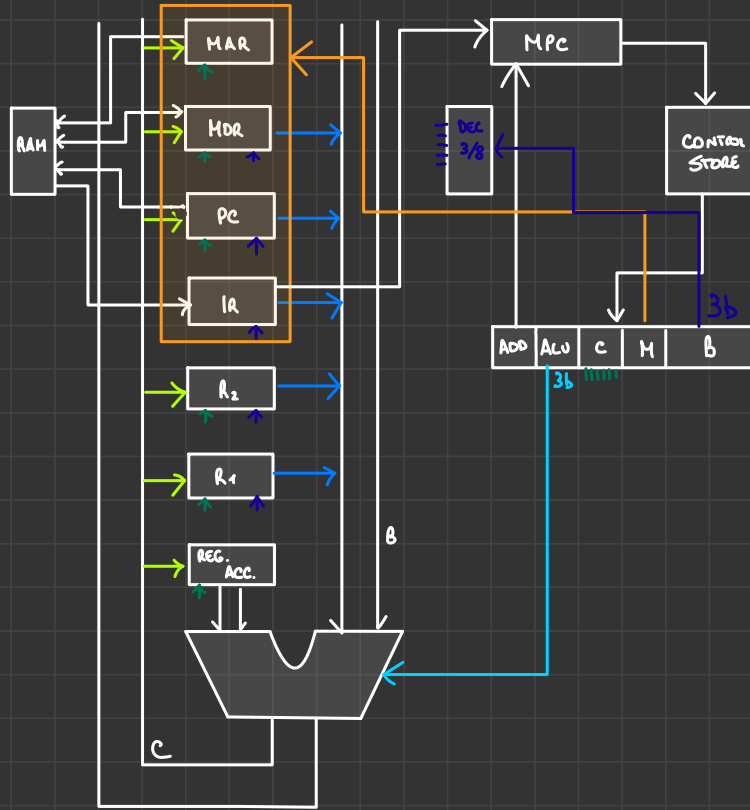
$$5:21:5$$

$$338 \cdot 8 = 264 \cdot 2^{24} \rightarrow \text{è troppo}$$

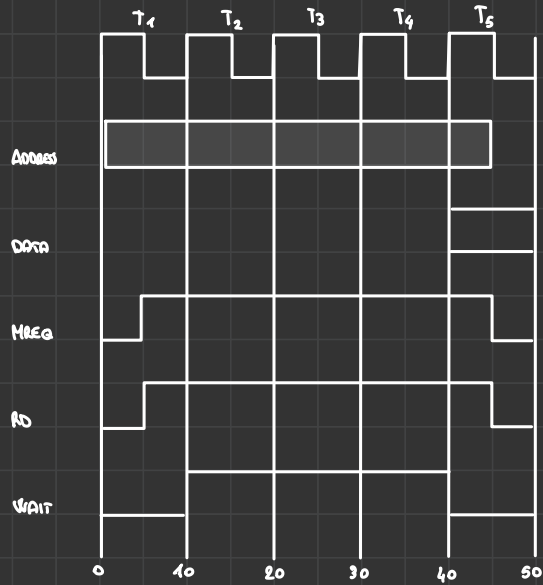
$$7:19:5$$

$$\textcircled{3} 2^7 \cdot 2^{-3} = 2^4 = 16$$

ARCHITETTURA CISC



$$T = \frac{1}{100 \cdot 10^6} = \frac{1000}{100} \cdot 10^{-3} = 10 \text{ ms}$$



① 30 ms

② 45 ms

③ 50 ms

④ 60 ms

⑤ 45 ms

⑥ 50 MHz 45 ms 20 ms

⑦ 40 ms

E SERCIZIO SU CACHE

8 GB (33 bit per gli indirizzi)

blocchi 64 B = 2^6

600 KB a disposizione

$$\textcircled{2} \quad 32k = 2^5 \cdot 2^{10} = 2^{15}$$

$$33 - 15 - 6 = 12$$

$$15:12:6$$

$$64B \cdot 2^{12} = 132 \cdot 2^{11} = 264 \cdot 2^{10} = 264 \text{ KB}$$

$\textcircled{3}$

$$600 \cdot 2^{10} = 300 \cdot 2^{11} = 150 \cdot 2^{12} = 15 \cdot 2^{13}$$

$$33 - 13 - 6 = 14$$

$$14:13:6$$

$$64B \cdot 2^{13} = 132 \cdot 2^{12} = 264 \cdot 2^{11} = 528 \text{ KB}$$

$\textcircled{3}$

$$1b: 14b: 64B$$

$\textcircled{3}$

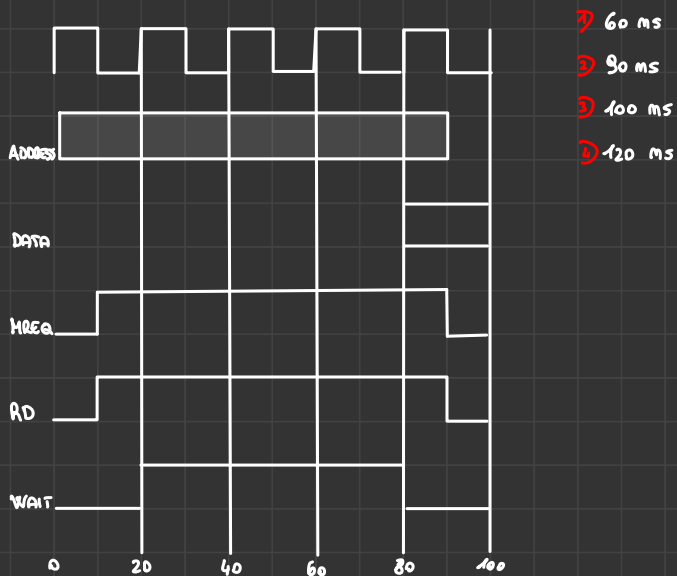
$$2^{14} = 16384$$

$$\textcircled{3} \quad 2^{13} \cdot 2^{-1} = 2^{12}$$

1. Esercizio su bus. Si consideri un bus sincrono che lavora a 50 Mhz con linee separate per dati e indirizzi dotato dei segnali di controllo MREQ, RD e WAIT e una memoria con un tempo di risposta di 80 nsec dal momento in cui gli indirizzi sono stabili.

- In condizioni ideali, per quanto tempo deve essere asserto il segnale di WAIT per garantire una lettura in memoria? Esprimere il risultato con un numero che indica il tempo in nsec.
- In condizioni ideali, dopo quanto tempo i dati saranno disponibili presso il master? Esprimere il risultato con un numero che indica il tempo in nsec.
- In condizioni ideali, quanto dura, complessivamente, una transazione di lettura? Esprimere il risultato con un numero che indica il tempo in nsec.
- In condizioni ideali, quanto dura, complessivamente, una transazione di lettura se la frequenza si dimezza? Esprimere il risultato con un numero che indica il tempo in nsec.

$$T = \frac{1}{50 \cdot 10^6} = \frac{1000}{50} \cdot 10^{-9} = 20 \text{ ns}$$



2. Esercizio su cache. Si vuole progettare una cache a mappatura diretta per un sistema a 22 bit e blocchi di 32 byte. Rispondere alle domande che seguono supponendo di avere a disposizione uno spazio di 100KB sul microprocessore.
- Indicare la dimensione minima della cache in grado di garantire che su ciascuna slot non collidano più di 128 blocchi. Esprimere il risultato nel formato XM (dove M=B,KB,MB,GB o TB) arrotondando all'intero superiore (per esempio: 125KB).
 - Indicare la struttura dell'indirizzo per la cache individuata nella domanda precedente, specificando la dimensione dei vari campi (tag,line,byte) nel formato X:Y:Z (per esempio: 5:12:8).
 - Indicare la struttura di una slot della cache individuata nella domanda precedente, specificando la dimensione dei vari campi (validità,tag,dati) nel formato XM:YM:ZM, dove M = b (bit) o B (Byte) (per esempio: 3b:15b:55B).
 - Indicare, indipendentemente dal numero di collisioni, la struttura dell'indirizzo che consente di massimizzare la dimensione della cache specificando la dimensione dei vari campi (tag,line,byte) nel formato X:Y:Z (per esempio: 5:12:8).
 - Indicare il numero di collisioni nel caso in cui, mantenendo lo stesso numero di slot della cache individuata nelle domande precedenti, la cache viene realizzata a due vie.

22 BIT

BLOCCHI 32 B = 2^5

100KB

$$\Rightarrow 128 = 2^7$$

$$22 - 7 - 5 = 10$$

V	TAG	BLOCCO
1b	7b	32b

$$32B \cdot 2^{10} = 33KB$$

$$\Rightarrow 4:10:5$$

$$\Rightarrow 1b:7b:32b$$

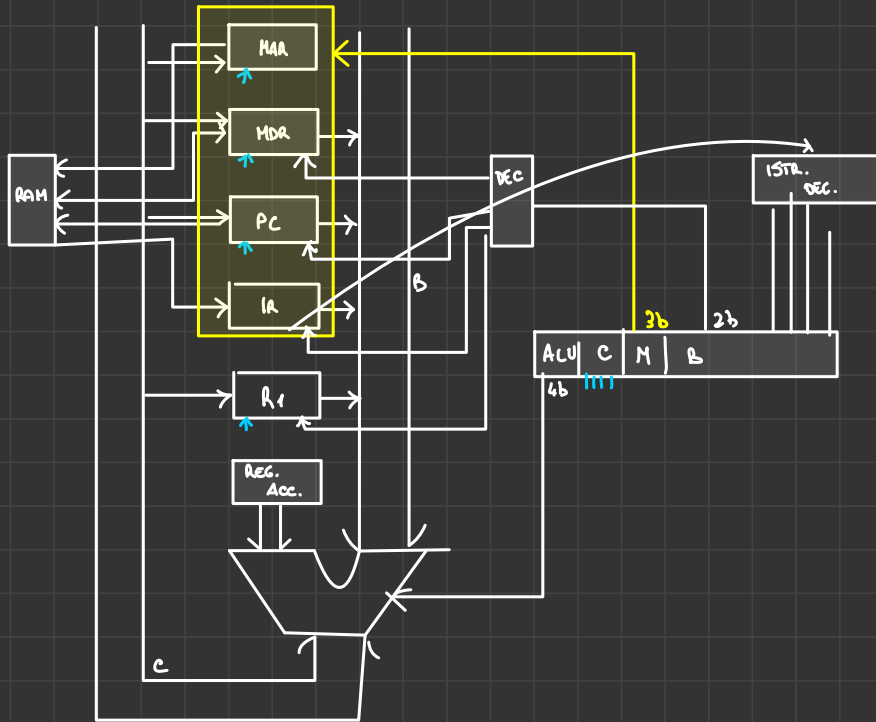
$$\Rightarrow 100KB = 100 \cdot 2^{10} = 50 \cdot 2^{11}$$

$$6:11:5$$

$$32B \cdot 2^{11} = 66KB$$

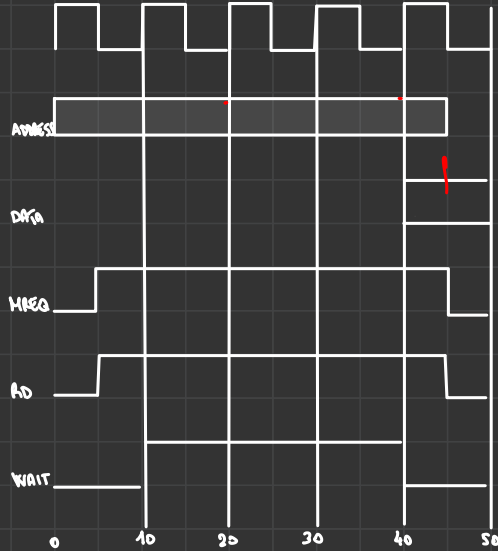
$$\Rightarrow 2^7 \cdot 2^{-1} = 2^6 = 64$$

ARCHITETTURA



HOMEWORK SUI BUS

$$T = \frac{1}{100 \cdot 10^6} = \frac{1000}{100} \cdot 10^{-9} = 10 \text{ ns}$$



③ 30 ms

③ 45 ms

③ 50 ms

③ 60 ms

③ 45 ms

③ 50 MHz

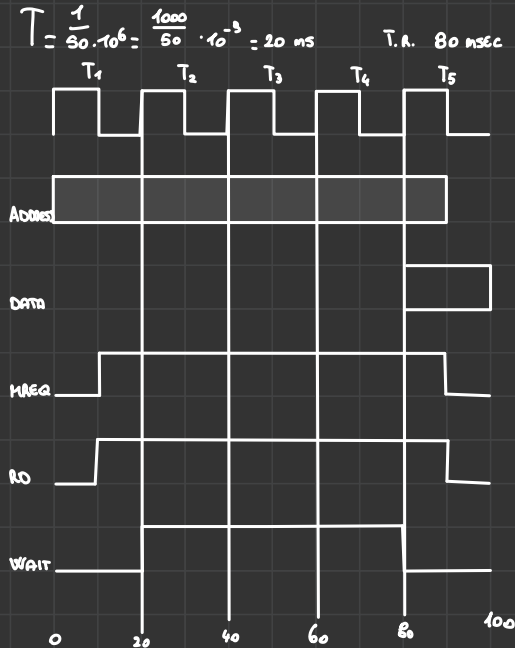
③ 70 ms

45 msec 20 ms

60 80

1. Esercizio su bus. Si consideri un bus sincrono che lavora a 50 Mhz con linee separate per dati e indirizzi dotato dei segnali di controllo MREQ, RD e WAIT e una memoria con un tempo di risposta di 80 nsec dal momento in cui gli indirizzi sono stabili.

- In condizioni ideali, per quanto tempo deve essere asserted il segnale di WAIT per garantire una lettura in memoria? Esprimere il risultato con un numero che indica il tempo in nsec.
- In condizioni ideali, dopo quanto tempo i dati saranno disponibili presso il master? Esprimere il risultato con un numero che indica il tempo in nsec.
- In condizioni ideali, quanto dura, complessivamente, una transazione di lettura? Esprimere il risultato con un numero che indica il tempo in nsec.
- In condizioni ideali, quanto dura, complessivamente, una transazione di lettura se la frequenza si dimezza? Esprimere il risultato con un numero che indica il tempo in nsec.



1. 60 ns

2. 80 ns

3. 100 ns

4. 120 ns

2. Esercizio su cache. Si vuole progettare una cache a mappatura diretta per un sistema a 22 bit e blocchi di 32 byte. Rispondere alle domande che seguono supponendo di avere a disposizione uno spazio di 100KB sul microprocessore.

- Indicare la dimensione minima della cache in grado di garantire che su ciascuna slot non collidano più di 128 blocchi. Esprimere il risultato nel formato XM (dove M=B,KB,MB,GB o TB) arrotondando all'intero superiore (per esempio: 125KB).
- Indicare la struttura dell'indirizzo per la cache individuata nella domanda precedente, specificando la dimensione dei vari campi (tag,line,byte) nel formato X:Y:Z (per esempio: 5:12:8).
- Indicare la struttura di una slot della cache individuata nella domanda precedente, specificando la dimensione dei vari campi (validità,tag,dati) nel formato XM:YM:ZM, dove M = b (bit) o B (Byte) (per esempio: 3b:15b:55B).
- Indicare, indipendentemente dal numero di collisioni, la struttura dell'indirizzo che consente di massimizzare la dimensione della cache specificando la dimensione dei vari campi (tag,line,byte) nel formato X:Y:Z (per esempio: 5:12:8).
- Indicare il numero di collisioni nel caso in cui, mantenendo lo stesso numero di slot della cache individuata nelle domande precedenti, la cache viene realizzata a due vie.

SISTEMA 22 BIT

BLOCCHI 32 BYTE

100KB

1.

$$128 = 2^7$$

$$22 - 7 - 5 = 10$$

$$7:10:5$$

$$32B \cdot 2^{10} = 32KB$$

2.

$$7:10:5$$

3.

$$1b:7b:32b$$

4.

$$100KB = 100 \cdot 2^{10} = 50 \cdot 2^{11}$$

$$22 - 11 - 5 = 6$$

$$6:11:5$$

$$32B \cdot 2^{11} = 66B \cdot 2^{10} = 66KB$$

5.

$$2^7 \cdot 2^1 = 2^6$$

INDIRIZZI 64 BIT

$$2GB \text{ RAM} = 2 \cdot 2^{30} = 2^{31}$$

$$\nearrow 2 \cdot 2^{40} = 2^{41}$$

CACHE DATA: 4 VIE 2K SLOT LINE 32 BYTE $\rightarrow 2^5$

CACHE ISTRUZIONI: 8 VIE

$$31 - 11 - 5 = 15$$

1. $15 : 11 : 5$

2. $1b : 15b : 32b : 34b \cdot 4 = 136b$

3. $1b : 15b : 32b, 1b : 15b : 32b, 1b : 15b : 32b, 1b : 15b : 32b$

4. $2^{15} \cdot 2^{-2} = 2^{13}$

6. $200MB = 200 \cdot 2^{20} = 100 \cdot 2^{21} = 50 \cdot 2^{22}$

$$31 - 22 - 5 = 4$$

$$4 : 22 : 5$$

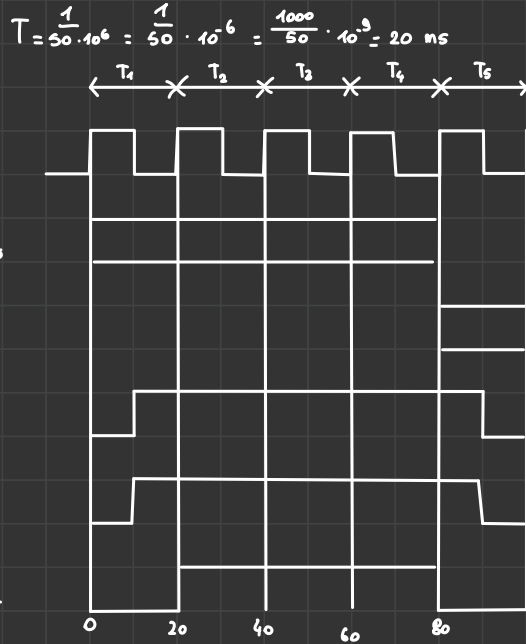
$$33b \cdot 8 = 264 \cdot 2^{22} =$$

$$31 - 18 - 5 = 7$$

$$7 : 18 : 5$$

$$2^7 \cdot 2^{-3} = 2^4$$

1. Esercizio su bus. Si consideri un bus sincrono che lavora a 50 Mhz con linee separate per dati e indirizzi dotato dei segnali di controllo MREQ, RD e WAIT e una memoria con un tempo di risposta di 80 nsec dal momento in cui gli indirizzi sono stabili.



1. 60 nSEC

2. 90 nSEC

3. 100 nSEC

4. 120 nSEC

2. Esercizio su cache. Si vuole progettare una cache a mappatura diretta per un sistema a 22 bit e blocchi di 32 byte. Rispondere alle domande che seguono supponendo di avere a disposizione uno spazio di 100KB sul microprocessore.

SISTEMA A 22 BIT

BLOCCHI DI 32 BYTE = 2^5

100KB SUL MICROPROCESSORE = $100 = 2^6$

TAG	LINE	BLOCCO
-----	------	--------

6b 11b 5b

V	TAG	BLOCCO
---	-----	--------

1b 6b 32b

A $128 = 2^7$

$22 - 7 - 5 = 10$ BIT PER INDICAZIONE 2^{10} SLOT ETICHETTA?

1 BIT VAL. 7 BIT. ETICHETTA 1 BLOCCO 32 BYTE

LA DIMENSIONE DI CIASCUN SLOT È 32 BYTE

$32 \cdot 2^{10} = 33 \text{ KB}$

D.

BLOCCHI 32B = $32 = 2^5$

100KB MICROPROC. $100 \cdot 2^{10} = 80 \cdot 2^{11}$

UNA CACHE AVENTE 2^{11} SLOT DA 32B MASSIMO

LINE

11b

$22 - 11 - 5 = 6$

6:11:5

E.

$2^{11} \cdot 2^5 = 2^{16}$

CONSIDERANDO 7:10:5 $2^7 = 64$

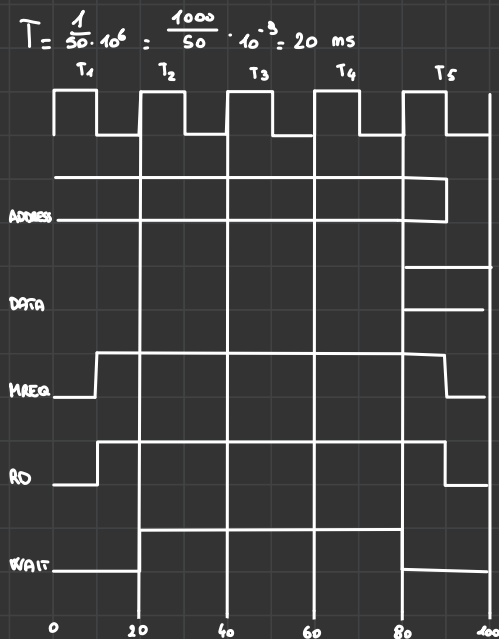
B 7:10:5

C.

1b:11b:32b

1. Esercizio su bus. Si consideri un bus sincrono che lavora a 50 Mhz con linee separate per dati e indirizzi dotato dei segnali di controllo MREQ, RD e WAIT e una memoria con un tempo di risposta di 80 nsec dal momento in cui gli indirizzi sono stabili.

- In condizioni ideali, per quanto tempo deve essere asserted il segnale di WAIT per garantire una lettura in memoria? Esprimere il risultato con un numero che indica il tempo in nsec.
- In condizioni ideali, dopo quanto tempo i dati saranno disponibili presso il master? Esprimere il risultato con un numero che indica il tempo in nsec.
- In condizioni ideali, quanto dura, complessivamente, una transazione di lettura? Esprimere il risultato con un numero che indica il tempo in nsec.
- In condizioni ideali, quanto dura, complessivamente, una transazione di lettura se la frequenza si dimezza? Esprimere il risultato con un numero che indica il tempo in nsec.



1. 60 msec.

2. 90 msec

3. 100 msec

4. 120 msec

2. Esercizio su cache. Si vuole progettare una cache a mappatura diretta per un sistema a 22 bit e blocchi di 32 byte. Rispondere alle domande che seguono supponendo di avere a disposizione uno spazio di 100KB sul microprocessore.

- Indicare la dimensione minima della cache in grado di garantire che su ciascuna slot non collidano più di 128 blocchi. Esprimere il risultato nel formato XM (dove M=B,KB,MB,GB o TB) arrotondando all'intero superiore (per esempio: 125KB).
- Indicare la struttura dell'indirizzo per la cache individuata nella domanda precedente, specificando la dimensione dei vari campi (tag,line,byte) nel formato X:Y:Z (per esempio: 5:12:8).
- Indicare la struttura di una slot della cache individuata nella domanda precedente, specificando la dimensione dei vari campi (validità,tag,dati) nel formato XM:YM:ZM, dove M = b (bit) o B (Byte) (per esempio: 3b:15b:55B).
- Indicare, indipendentemente dal numero di collisioni, la struttura dell'indirizzo che consente di massimizzare la dimensione della cache specificando la dimensione dei vari campi (tag,line,byte) nel formato X:Y:Z (per esempio: 5:12:8).
- Indicare il numero di collisioni nel caso in cui, mantenendo lo stesso numero di slot della cache individuata nelle domande precedenti, la cache viene realizzata a due vie.

2. $7:10:5$

3. $1b:7b:32B$

4. $6:11:5$

5. $2^{11} \cdot 2^1 \cdot 2^{10}$

22 BIT TOTALI

32 B BLOCCHI = 2^5 5 BIT 1 BLOCCO

100KB A DISP. = $100 \cdot 2^{10} = 50 \cdot 2^{11}$

1.

128 = 2^7 TAG

32 B 1 BLOCCO 2^5 $22 - 7 - 5 = 10$

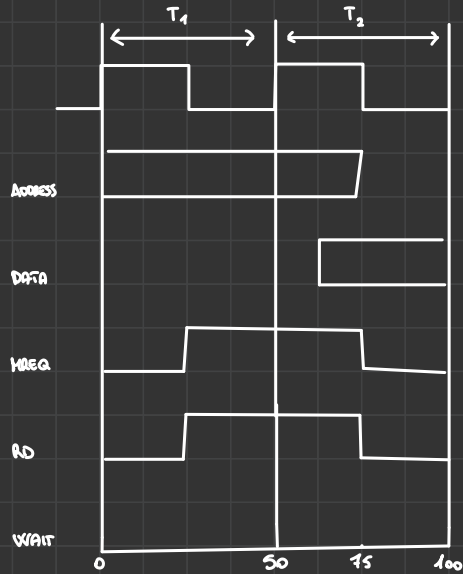
V	TAG	Blocco
7b		32 B

TAG	SLOT	BYTE
7	10	5

$\rightarrow 1 \text{ BLOCCO} = 32 \text{ B} \cdot 2^{10} = 33 \text{ KB}$

SECONDO ESAME 2021

$$T = \frac{1}{20 \cdot 10^6} = \frac{1000}{20} \cdot 10^{-9} = 50 \text{ ns}$$



1. 0
2. 75 ns
3. 100 ns
4. 15 ns
5. 125 ns
6. 200 ns

ESERCIZIO SULLA CACHE

64 bit

RAM $2GB = 2 \cdot 2^{30} = 2^{31}$

SE NON MI DA
LA LUNGHEZZA DELL'
INDIRIZZO LA PRENDO
DALLA MEMORIA CENTRALE

CACHE DATA 4 VIE 2K SLOT 32 BYTE

CACHE ISTRUZIONI 8 VIE 32 BYTE

1.

SLOT 11b $31 - 11 - 5 = 15$

Blocco 5b

15:11:5

2.

V	LINE	Blocco
1b	15b	32b

$34B \cdot 4 = 136B$

3.

1b:15b:32b

4. $2^{15} \cdot 2^2 = 2^{17}$
 4 VIE

5.

1k

6. 200 MB (32 byte)

$200 \cdot 2^{20}$

$31 - 20 - 5 = 6$

$6:20:5$

1b:6b:32b $\approx 33B$

$33 \cdot 8 = 264B \cdot 2^{20} = 264MB \rightarrow$ PIU' GRANDE DI 200
QUINDI DIMINUISCO SLOT
E AUMENTO IL TAG

7:13:5

1b:7b:32b

$33B \cdot 8 = 264B \cdot 2^{13} = 132 \cdot 2^{20} = 132MB \checkmark$

7.

2^7
 PESO LA
CACHE È A 8 VIE $2^7 - 2^3 = 2^4$

Esercizi su Assembler. Con riferimento al seguente programma assembler, rispondere alle

```
1 _EXIT = 1 → ROUTINE DI SOSTA
2 _PRINTF = 127 → ROUTINE PER LA STAMPA
3 .SECT .TEXT
4 start:
5     MOV BP,SP → Svuoto la pila
6     PUSH n-v → tocca la dimensione dell'array in byte
7     PUSH (n)
8     PUSH v → riempio la pila
9     PUSH 0
10    CALL beta
11    POP AX
12    MOV SP,BP
13    PUSH AX
14    PUSH fmt
15    PUSH _PRINTF
16    SYS
17    MOV SP,BP
18    PUSH 0
19    PUSH _EXIT
20    SYS
21 beta:
22    PUSH BP } → AZIONI CHE VANNO
23    MOV BP,SP } → SEMPRE FATTE QUANDO USO CALL
24    MOV DI,6(BP)
25    MOVB AL,8(BP)
26    MOV CX,10(BP)
27 1: SCASB
28    JNE 2f
29    INC 4(BP)
30 2: LOOP 1b
31    POP BP
32    RET
33 .SECT .DATA
34 v: .BYTE 3,4,7,11,3 → VETTORE DI 5 EL.
35 n: .BYTE 3 → VAL. CON 3 ALI' NUMERO
36 fmt: .ASCII "%d" → STAMPA
37 .SECT .BSS
38
```

E SAME 23 SETTEMBRE 2020

$$\text{MEM } 4\text{MB} = 4 \cdot 2^{20} = 2^{22}$$

BLOCKHI 32B

100kB A DISP.

1.

$$128 = 2^7 \rightarrow \text{TAG}$$

BLOCKHI 2^5

$$22 - 5 - 7 = 10$$

V	TAG	BLOCK
1b	7b	32B

$$33\text{B} \cdot 2^{10} = 33\text{kB}$$

2.

7:10:5

3.

1b:7b:32B

4.

$$100\text{kB} = 100 \cdot 2^{10} = 50 \cdot 2^{11}$$

BLOCK 5

$$22 - 11 - 5 = 6$$

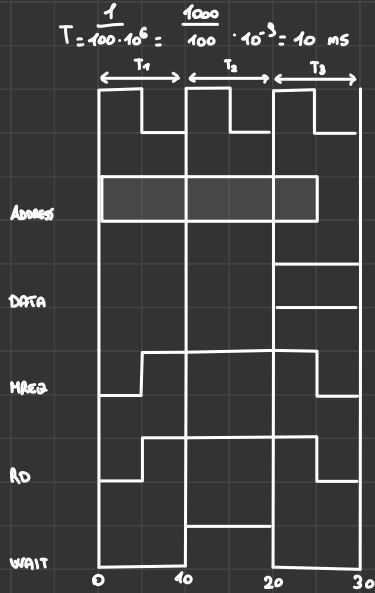
6:11:5

5.

$$2^{11}$$

/

ESAME LUGLIO 2018



1. 10 ms
2. 25 ms
3. 30 ms
4. 40 ms

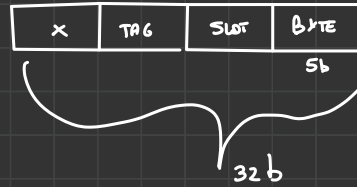
ESERCIZIO SULLA CACHE

32 BIT

32 BYTE CACHE LINE = 2^5

4 MB MEMORIA = $2^2 \cdot 2^{20} = 2^{22}$

40 KB MEM. DISP 22 b



ANDIAMO PER TENTATIVI

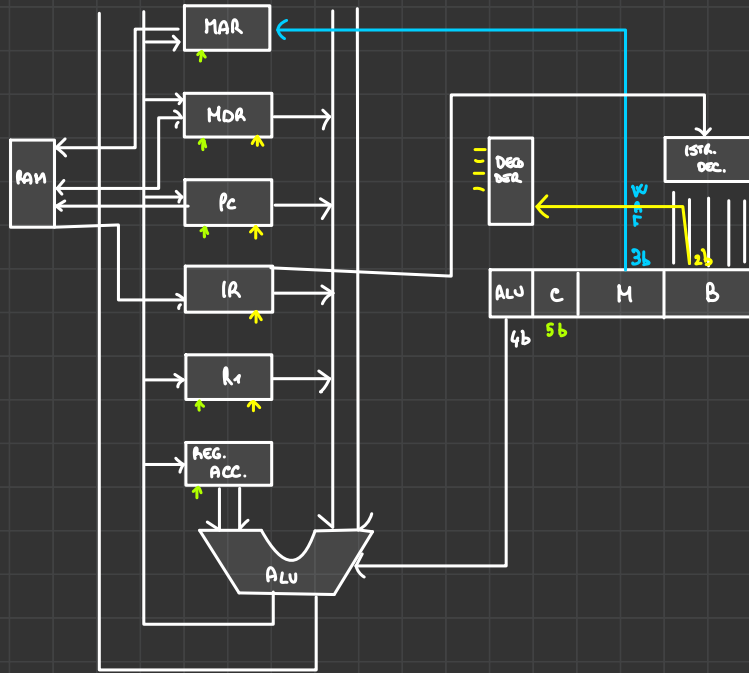
TAG = 7 b Slot = $22 - 7 - 5 = 10$

1b:7b:32b

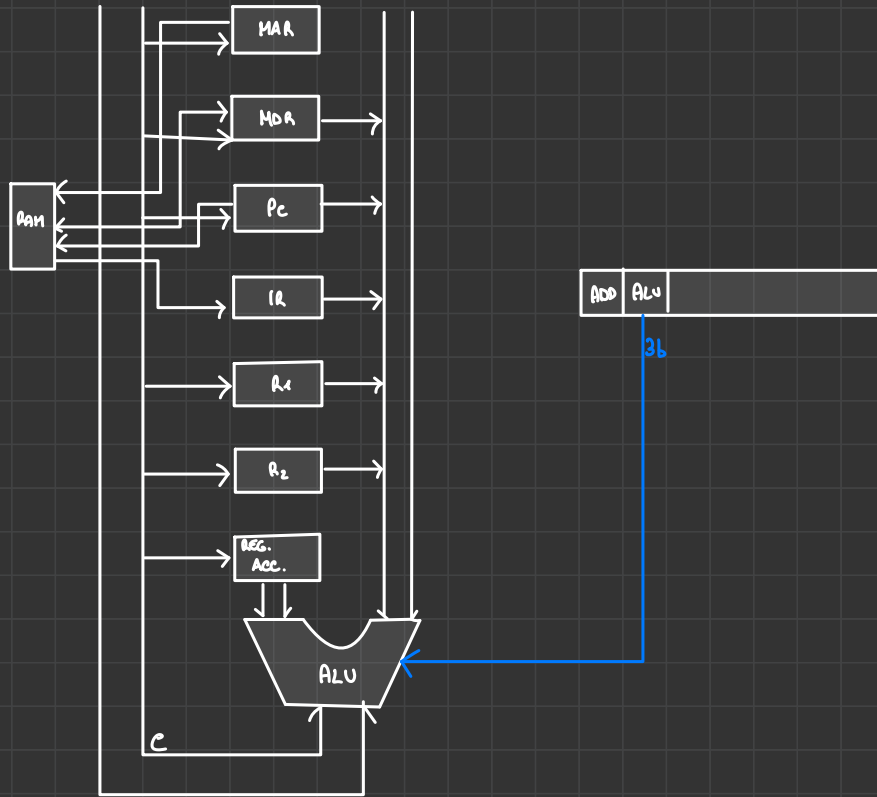
$32 \text{ b} \cdot 2^{10} = 33 \text{ KB}$ ✓

Esercizio su Modellazione RISC

ATTENZIONE: IL BUS C NON COLLEGA IA



Esercizio su Modellazione CISC



SECONDO ESERCIZIO 2022

INDIRIZZI A 64 BIT

$$2GB \text{ DI RAM} = 2 \cdot 2^{30} = 2^{31}$$

CACHE DATA 4 VIE 2K SLOT = 2^{11} CACHE LINE 32 BYTE 2^5 TAG = $31 - 11 - 5 = 15$

CACHE ISTRUZIONI 8 VIE

CACHE DATA

TAG	SLOT	BYTE
15b	11b	5b

15:11:5

✓	TAG	LINE
1b	15b	32b

$$= 32b \cdot 4 = 128b$$

$$1b:15b:32b, 1b:15b:32b, 1b:15b:32b, 1b:15b:32b$$

COLLISIONI $2^{15} \cdot 2^2 = 2^{17}$ ↑ 4 VIE

5.

TAG 13 BIT BLOCCHI 2^5

$$31 - 13 - 5 = 13$$

TAG	SLOT	BYTE
13b	13b	5b

6.

$$200 MB = 200 \cdot 2^{20} = 100 \cdot 2^{21} = 50 \cdot 2^{22}$$

$$31 - 22 - 5 = 4$$

1b	4b	32b
----	----	-----

$$31 - 13 - 5 = 7$$

$$33b \cdot 8 = 264 \cdot 2^{13} = 132 \cdot 2^{20} = 132 MB$$

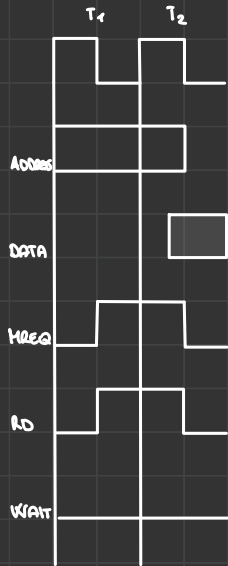
$$7:13:5$$

$$33b \cdot 8 = 264 \cdot 2^{22}$$

TROPPO GRANDE

$$2^7 \cdot 2^3 = 2^{10}$$

$$T = \frac{1}{20 \cdot 10^6} = \frac{1000}{20} \cdot 10^{-9} = 50 \text{ ns}$$



③ 0 ms

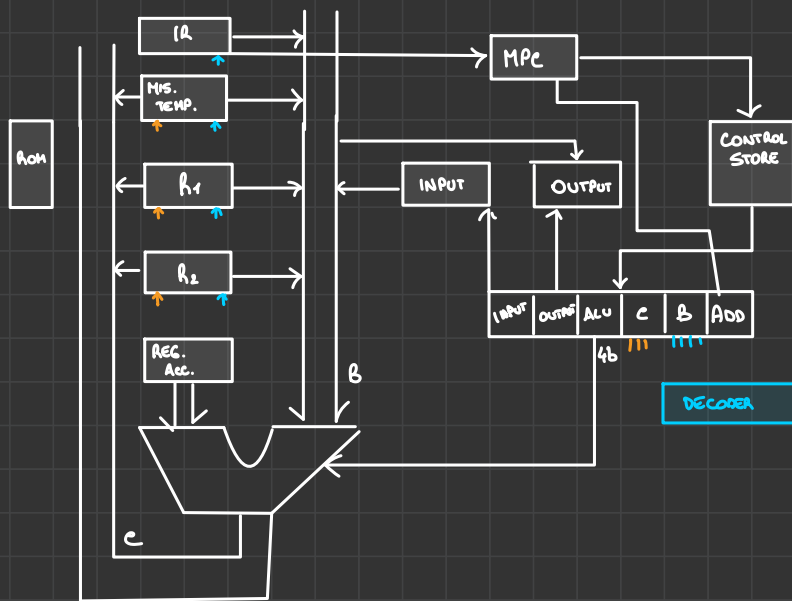
③ 75 ms

③ 100 ms

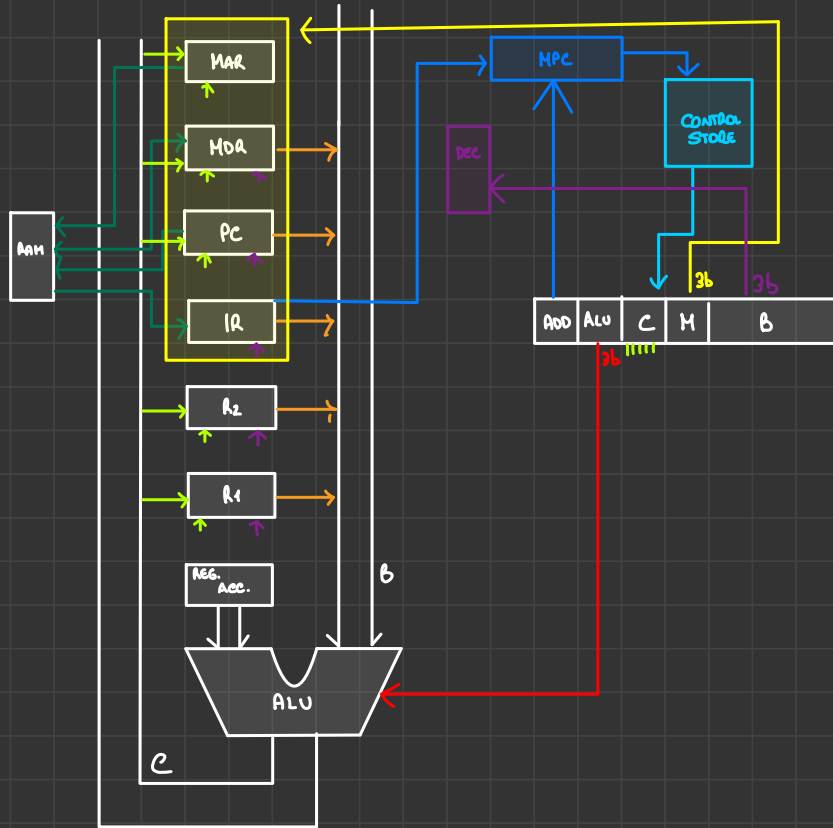
③ Se la frequenza si raddoppia si dimezza il periodo, di conseguenza i dati saranno disponibili presso il master a metà del terzo periodo e l'intera transazione di lettura durerà 3 periodi (75ns)

③ MEM. T.R. 85 ms QUINDI ENTRA IN GIOCO IL TERZO PERIODO QUINDI 125 ms

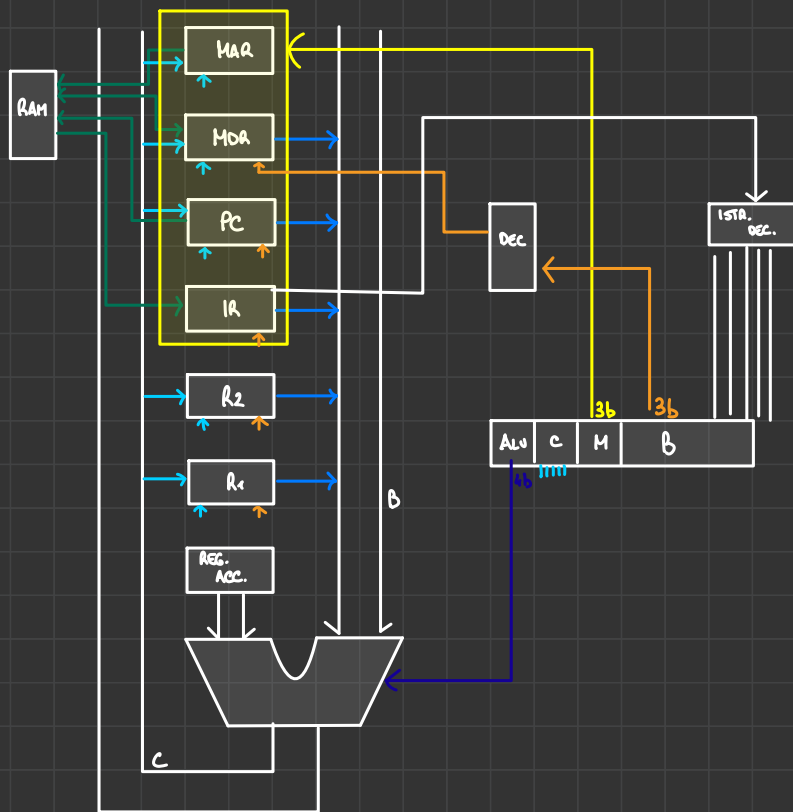
③ 200 ms



MODELLAZIONE ARCHITETTURA CISC



MODELLAZIONE ARCHITETTURA RISC



!Calcola la lunghezza della stringa e il numero di occorrenze di un determinato carattere

```
_EXIT = 1
_PRINTF = 126
```

```
.SECT .TEXT
start:
MOV BX, end
PUSH s
CALL LUN
MOV SP, BP
PUSH s
PUSH carattere
CALL OCC
```

```
MOV SP, BP
PUSH 0
PUSH _EXIT
SYS
```

```
LUN:
PUSH BP
MOV BP, SP
MOV CX, (BX)-s
SHR CX, 1
1: ADD AX, 1
LOOP 1b
PUSH AX
PUSH format
PUSH _PRINTF
POP BP
SYS
RET
```

```
OCC:
PUSH BP
MOV BP, SP
MOV AX, 0
MOV SI, 0
begin_loop:
MOV BX, 6(BP)
MOV DX, (BX)(SI)
MOV BX, 4(BP)
CMPB BL, DL
JE 1f
JMP 2f
1: INC AX
2: INC SI
LOOP begin_loop
```

```
PUSH AX
PUSH carattere
PUSH _FORMAT
PUSH _PRINTF
POP BP
SYS
RET
```

```
.SECT .DATA
s: .ASCII "ciao"
end: .WORD 1
carattere: .ASCII "c"
format: "La lunghezza della stringa e' %d"
_FORMAT: "Il numero di occorrenze di %c e' %d"
```

```
.SECT .BSS
```